

Projeto 1 — LED com Push Button

Objetivo

Montar um circuito simples com:

- 1 LED
- 1 push button
- Arduino
- resistores

Esse primeiro projeto serve para entender:

- o que é **entrada** (input) → botão
- o que é **saída** (output) → LED
- como o Arduino **lê e responde**

1. Materiais

Você vai precisar de:

- 1 Arduino Uno
- 1 LED
- 1 push button
- 1 resistor de **220Ω** para o LED
- 1 resistor de **10kΩ** para o botão
- jumpers
- protoboard

2. Entendendo cada componente

LED

É a saída visual do projeto.

Quando o Arduino mandar energia para ele, ele acende.

Push Button

É um botão momentâneo.

Ele só envia sinal enquanto estiver sendo pressionado.

Resistor de 220Ω

Protege o LED para ele não queimar.

Resistor de 10kΩ

Serve para estabilizar a leitura do botão, evitando leitura errada.

4. Como esse circuito funciona

Quando o botão **não está pressionado**, o resistor de 10kΩ “puxa” o pino 8 para o GND, então o Arduino entende:

LOW

Quando o botão **é pressionado**, o pino 8 recebe 5V, então o Arduino entende:

HIGH

Ou seja:

- **solto** = LOW
- **pressionado** = HIGH

5. Código base do projeto

```
int led = 12; // LED ligado no pino 12
```

```
int botao = 8; // Botão ligado no pino 8
```

```
void setup() {
```

```
    pinMode(led, OUTPUT); // Define o LED como saída
```

```
    pinMode(botao, INPUT); // Define o botão como entrada
```

```
}
```

```
void loop() {
```

```
    int estadoBotao = digitalRead(botao); // Lê o estado do botão
```

```
    if (estadoBotao == HIGH) { // Se o botão for pressionado
```

```
    digitalWrite(led, HIGH); // Acende o LED
} else {          // Senão
    digitalWrite(led, LOW); // Apaga o LED
}
}
```

6. Explicando o código linha por linha

Declaração das variáveis

```
int led = 12;
int botao = 8;
```

Aqui estamos dizendo:

- o LED está no pino 12
- o botão está no pino 8

Função setup()

```
void setup() {
    pinMode(led, OUTPUT);
    pinMode(botao, INPUT);
}
```

Essa parte roda só uma vez, quando o Arduino liga.

- OUTPUT = saída
- INPUT = entrada

Então:

- LED será controlado pelo Arduino
- botão será lido pelo Arduino

Função loop()

```
void loop() {
```

Tudo que está aqui dentro fica repetindo sem parar.

Leitura do botão

```
int estadoBotao = digitalRead(botao);
```

O Arduino lê o estado do pino 8.

Ele pode retornar:

- HIGH
- LOW

Condição

```
if (estadoBotao == HIGH) {  
    digitalWrite(led, HIGH);  
} else {  
    digitalWrite(led, LOW);  
}
```

Se o botão estiver pressionado:

- acende o LED

Senão:

- apaga o LED

7. O que você precisa entender aqui

Nesse projeto, o Arduino fica observando o botão o tempo todo. Quando detecta que ele foi pressionado, envia energia para o LED acender. Quando o botão é solto, ele corta esse sinal e o LED apaga.

Esse projeto ensina a lógica:

“Se acontecer uma entrada, eu executo uma saída.”

No nosso caso:

- **entrada:** pressionar botão
- **saída:** acender LED

Isso é a base de praticamente todos os projetos de automação.

Projeto 2 — Acender LED com pressão de 1 segundo

Objetivo:

O LED só deve acender se o botão ficar pressionado por **pelo menos 1 segundo**
Caso contrário, o LED permanece apagado

O que vamos aprender aqui

- Leitura de botão (digitalRead)
- Uso de **tempo (delay)**
- Lógica condicional (if)
- Controle baseado em **tempo de ação**

Ideia da lógica

O Arduino vai fazer:

SE botão pressionado

espera 1 segundo

SE ainda estiver pressionado

acende LED

SENÃO

não faz nada

Aqui entra um conceito MUITO importante:

“Confirmar a ação depois de um tempo”

Código base do projeto

```
int led = 12; // LED ligado no pino 12
```

```
int botao = 8; // Botão ligado no pino 8
```

```
void setup() {
```

```
  pinMode(led, OUTPUT); // Define o LED como saída
```

```
  pinMode(botao, INPUT); // Define o botão como entrada
```

```
}
```

```
void loop() {  
  
  int estadoBotao = digitalRead(botao); // Lê o estado do botão  
  
  if (estadoBotao == HIGH) { // Se o botão for pressionado  
    delay(1000);      // Espera 1 segundo  
    digitalWrite(led, HIGH); // Acende o LED  
  } else {           // Senão  
    digitalWrite(led, LOW); // Apaga o LED  
  }  
}
```

Não basta apertar... tem que segurar.

Isso cria um comportamento tipo:

- botão rápido → não funciona
- botão pressionado → funciona

Nesse projeto, o Arduino só responde se a ação for mantida por um tempo. Isso simula situações reais, como segurança, confirmação ou intenção do usuário.

Projeto 3 — Contar quantas vezes o botão foi pressionado

Objetivo:

Cada vez que o botão for pressionado:

- O Arduino **conta +1**
- Mostra no **Serial Monitor**

O que o voce aprende aqui

- Variáveis (memória)
- Incremento (+1)
- Detecção de clique (evento)
- Problema de múltiplas leituras (bounce)

Conceito MUITO IMPORTANTE

Se você fizer assim:

```
if (botao == HIGH) {  
    contador++;  
}
```

Vai contar MUITAS vezes com um único clique porque o Arduino é rápido demais

Solução: detectar o “momento do clique”

A gente precisa detectar quando o botão **acabou de ser pressionado**

Estratégia

Guardar o estado anterior do botão:

- Antes → LOW
 - Agora → HIGH
- Isso significa: **clique**

Código base do projeto

```
int botao = 8; // Declara o pino onde o botão está conectado (pino digital 2)
```

```
int estadoAtual = 0; // Variável que guarda o estado atual do botão (HIGH ou LOW)
```

```
int estadoAnterior = 0; // Variável que guarda o estado anterior do botão
```

```
int contador = 0; // Variável que conta quantas vezes o botão foi pressionado
```

```
void setup() {  
    pinMode(botao, INPUT);  
    Serial.begin(9600);  
}  
  
void loop() {  
  
    // Lê o estado atual do botão (pressionado = HIGH, solto = LOW)  
    estadoAtual = digitalRead(botao);  
  
    // Verifica se houve um clique  
    // Ou seja: antes estava LOW (solto) e agora está HIGH (pressionado)  
    if (estadoAtual == HIGH && estadoAnterior == LOW) {  
  
        contador++; // Incrementa o contador (soma +1 a cada clique)  
        Serial.print("Contador: "); // Mostra a palavra "Contador: " no monitor serial  
        Serial.println(contador); // Mostra o valor atual do contador  
  
        delay(200); // Pequeno atraso para evitar múltiplas leituras do mesmo clique  
                    (debounce)  
    }  
  
    // Atualiza o estado anterior com o valor atual  
    // Isso é essencial para detectar a mudança no próximo loop  
    estadoAnterior = estadoAtual;  
}
```

Explicação

1. Variável contador


```
int contador = 0;
```

Guarda quantas vezes clicou

2. Leitura atual

```
estadoAtual = digitalRead(botao);
```

3. Comparação importante

```
if (estadoAtual == HIGH && estadoAnterior == LOW)
```

Isso significa:

Antes Agora Resultado

LOW HIGH Clique detectado

4. Incremento

```
contador++;
```

Mesma coisa que:

```
contador = contador + 1;
```

5. Mostrar no Serial

```
Serial.println(contador);
```

Aparece no monitor serial

6. Atualiza o estado

```
estadoAnterior = estadoAtual;
```

Prepara para a próxima leitura

Eu só conto quando o botão sai do 'não pressionado' para 'pressionado'

Problema real: Bounce (rebote)

Quando você aperta o botão:

Ele pode gerar vários sinais rápidos

Por isso usamos:

`delay(200);`

para evitar contagens duplicadas

Projeto 4 — Acender com 3 cliques e apagar com 3 cliques

Objetivo

Fazer o Arduino contar os cliques no botão.

A regra será:

- ao completar **3 cliques**, o LED muda de estado
- se estiver apagado, acende
- se estiver aceso, apaga

Ideia da lógica

O Arduino vai fazer isso:

1. detectar cada clique
2. somar no contador
3. quando chegar em 3:
 - trocar o estado do LED
 - zerar o contador de cliques

Código base do projeto

```
// Define o pino onde o botão está conectado (pino digital 8)
int botao = 8;
```

```
// Define o pino onde o LED está conectado (pino digital 12)
```

```
int led = 12;

// Variável que armazena o estado atual do botão (HIGH ou LOW)
int estadoAtual = 0;

// Variável que armazena o estado anterior do botão
int estadoAnterior = 0;

// Variável que conta quantos cliques foram dados no botão
int contadorClique = 0;

// Variável booleana que guarda o estado do LED (true = ligado, false = desligado)
bool estadoLed = false;

void setup() {

// Define o pino do botão como entrada
pinMode(botao, INPUT);

// Define o pino do LED como saída
pinMode(led, OUTPUT);

// Inicia a comunicação serial para exibir informações no monitor serial
Serial.begin(9600);
}

void loop() {

// Lê o estado atual do botão (pressionado = HIGH, solto = LOW)
estadoAtual = digitalRead(botao);

// Verifica se houve um clique (transição de LOW para HIGH)
if (estadoAtual == HIGH && estadoAnterior == LOW) {

// Incrementa o contador de cliques
contadorClique++;

// Exibe no monitor serial a quantidade de cliques
Serial.print("Clique: ");
Serial.println(contadorClique);
}
```

```

// Verifica se já foram dados 3 cliques
if (contadorCliques == 3) {

    // Inverte o estado do LED (se estava desligado liga, se estava ligado desliga)
    estadoLed = !estadoLed;

    // Atualiza o estado do LED no pino digital
    digitalWrite(led, estadoLed);

    // Verifica se o LED está ligado
    if (estadoLed == true) {
        Serial.println("LED ACESO");
    } else {
        // Caso contrário, o LED está desligado
        Serial.println("LED APAGADO");
    }

    // Reinicia o contador de cliques para começar novamente
    contadorCliques = 0;
}

// Pequeno atraso para evitar múltiplas leituras do mesmo clique (debounce)
delay(200);
}

// Atualiza o estado anterior com o estado atual para a próxima leitura
estadoAnterior = estadoAtual;
}

```

Explicação linha por linha

1. Variáveis principais

```

int contadorCliques = 0;
bool estadoLed = false;

```

contadorCliques

Guarda quantos cliques já aconteceram.

estadoLed

Guarda se o LED está:

- false → apagado

- true → aceso

2. Detectar clique

if (estadoAtual == HIGH && estadoAnterior == LOW)

Isso quer dizer:

- antes o botão não estava pressionado
- agora ele foi pressionado

Então tivemos **um clique**

3. Somar os cliques

contadorClique++;

Cada clique soma 1.

4. Verificar se chegou em 3

if (contadorClique == 3)

Quando atingir 3, o Arduino executa a ação.

5. Inverter estado do LED

estadoLed = !estadoLed;

Essa linha é muito interessante.

Ela faz o seguinte:

- se estadoLed era false, vira true
- se era true, vira false

Ou seja:

- apagado → acende
- aceso → apaga

6. Aplicar no pino

```
digitalWrite(led, estadoLed);
```

Se estadoLed for true, escreve HIGH

Se estadoLed for false, escreve LOW

7. Zerar a contagem

```
contadorCliques = 0;
```

Depois de completar 3 cliques, o contador volta para zero para começar outra rodada.

O Arduino vai juntando os cliques. Quando fecha um grupo de 3, ele troca o estado do LED. Depois começa a contar tudo de novo.

Exemplo prático

Situação inicial

- LED apagado
- contador = 0

Cliques

1º clique

- contador = 1
- LED continua apagado

2º clique

- contador = 2
- LED continua apagado

3º clique

- contador = 3
- LED acende
- contador volta para 0

4º clique

- contador = 1

5º clique

- contador = 2

6º clique

- contador = 3
- LED apaga
- contador volta para 0

O que você aprende aqui

- contagem de eventos
- lógica por condição
- memória de estado
- alternância de saída

Isso já parece um sistema real de automação.

ATIVIDADE — LED controlado por número de cliques (PAR / ÍMPAR)

Objetivo

Desenvolver um sistema onde:

- Cada clique no botão é contado
- Se o número de cliques for **PAR** → **LED acende**
- Se for **ÍMPAR** → **LED apaga**

Competências trabalhadas

- Leitura de entrada digital (digitalRead)
- Controle de saída (digitalWrite)
- Uso de variáveis
- Operador matemático % (módulo)
- Lógica condicional (if / else)
- Detecção de evento (clique)

Materiais

- 1 Arduino
- 1 push button
- 1 LED
- 1 resistor 220 Ω (LED)
- 1 resistor 10k Ω (botão)
- Protoboard + jumpers

Montagem

- Botão $\rightarrow$ pino **8**
- LED $\rightarrow$ pino **12**

Situação-problema (contextualização)

Imagine que você está desenvolvendo um sistema onde:

- cada clique representa uma ação do usuário
- o sistema precisa indicar estados diferentes

O LED deve funcionar como um indicador:

- número PAR $\rightarrow$ sistema ativo (LED ligado)
- número ÍMPAR $\rightarrow$ sistema inativo (LED desligado)